

Informe Final del Compilador de HULK

Frontend con Flex/Bison, análisis semántico, generación de bytecode, máquina virtual de pila y extensión de lenguaje mediante macros

Ronald Alfonso Pérez , Abraham Rey Sánchez Amador, Rodrigo Meseros González

Proyecto de Compilación / Lenguajes de Programación
Universidad de La Habana

22 de junio de 2026

Resumen

Este informe presenta el diseño y la implementación de un compilador para HULK construido en C++ como un sistema por fases: análisis léxico generado con Flex, análisis sintáctico generado con Bison en modo GLR, construcción de un AST, una fase de expansión de macros y desazucarado, análisis semántico en varias pasadas, generación de un bytecode propio y una máquina virtual de pila que ejecuta dicho bytecode. El énfasis del documento no está solo en enumerar componentes del repositorio, sino en justificar las decisiones de diseño tomadas en cada frontera del compilador: por qué se usan generadores (Flex/Bison) en lugar de un analizador manual, por qué el parser se declara GLR, por qué las macros y `match` se resuelven como transformaciones AST→AST antes de la semántica, por qué se define un bytecode propio en lugar de emitir ensamblador o LLVM, y por qué la ejecución se delega a una máquina virtual de pila enlazada a través de un artefacto intermedio. Desde la perspectiva de Lenguajes de Programación, la extensión propia principal es un *sistema de macros sintácticas* con parámetros por valor y de bloque, higiene mediante `$`-placeholders y expansión recursiva acotada, complementado por la construcción `match` con desazucarado a primitivas existentes. El objetivo del informe es presentar de forma formal el alcance técnico del compilador, las garantías estáticas que ofrece, los compromisos asumidos y las limitaciones que quedan como trabajo futuro. El sistema supera el 100 % de la batería de pruebas de entrega final (115/115).

Índice

1. Introducción	4
2. Alcance del compilador	4

2.1. Propósito general	4
2.2. Lenguaje soportado	5
2.3. Alcance técnico	5
2.4. Pipeline de alto nivel	6
3. Frontend	7
3.1. Decisión léxica: lexer generado con Flex	7
3.2. Decisión sintáctica: Bison en modo GLR	8
3.3. AST como frontera entre sintaxis y semántica	8
4. Análisis semántico	9
4.1. Análisis en varias pasadas	9
4.2. Tabla de tipos, subtipado y protocolos	10
4.3. Inferencia acotada	10
4.4. Resolución de nombres, self y base	10
4.5. Transpilación de lambdas a funtores	10
4.6. Errores semánticos diferenciados	11
4.7. Funciones y constantes incorporadas	11
5. Generación de bytecode	11
5.1. Motivación de un bytecode propio	11
5.2. Modelo de pila y conjunto de instrucciones	12
5.3. Lowering de objetos, métodos y base	12
5.4. Lowering de self , vectores, ciclos y lambdas	12
5.5. Serialización	13
6. Máquina virtual de ejecución	13
6.1. Arquitectura	13
6.2. Modelo de valores y objetos	13
6.3. Llamadas, despacho y builtins	14
6.4. Por qué la VM no es “otro backend”	14
7. Extensión propia de lenguaje: macros y match	14

7.1. Problema de diseño	14
7.2. Sintaxis y representación	14
7.3. Expansión, higiene y recursión	15
7.4. La construcción <code>match</code>	15
7.5. Comparación con otros lenguajes	16
8. Característica adicional: comprensión de listas y generadores perezosos	16
8.1. Comprensión de listas	16
8.2. Generadores perezosos	17
8.3. Relación entre ambas características	18
9. Validación y resultados	18
10. Construcción y reproducibilidad	19
11. Limitaciones y trabajo futuro	19
12. Conclusiones	20

1. Introducción

Construir un compilador para HULK no consiste solamente en transformar texto de entrada en otro texto de salida. El objetivo central es preservar el significado de un programa a través de varias capas de representación. Un programa fuente con expresiones, funciones, objetos, herencia, protocolos, vectores, iterables, lambdas y una extensión como las macros debe convertirse gradualmente en estructuras internas verificables y, finalmente, en una forma ejecutable.

El sistema implementa una arquitectura por fases. Primero, el frontend reconoce tokens y estructura sintáctica; después, el AST elimina detalles de la gramática concreta; luego, una fase de expansión transforma macros y `match` en construcciones del núcleo del lenguaje; a continuación, la fase semántica resuelve nombres, ámbitos, tipos, atributos, métodos, `self`, `base` y builtins, y transpila las lambdas a tipos funtores; más adelante, el generador de código produce un bytecode propio; finalmente, ese bytecode se serializa y se ejecuta sobre una máquina virtual de pila.

El compilador se materializa en dos ejecutables contruidos con CMake: `hulk`, que realiza todas las fases de compilación hasta producir el bytecode, y `hulk-vm`, que ejecuta el bytecode serializado. Esta separación es importante desde el punto de vista académico: el sistema no se comporta como una caja negra, sino como un conjunto de transformaciones observables, donde el artefacto intermedio entre compilador y máquina virtual puede inspeccionarse.

Este informe no se organiza como una descripción cronológica del repositorio ni como una lista de archivos. Su estructura sigue una idea de diseño: en cada fase se explica qué problema debía resolverse, qué alternativas existían, qué decisión se adoptó y qué consecuencias tiene esa decisión para las fases posteriores. La arquitectura en C++ se presenta como soporte físico de la implementación; el centro del análisis son las decisiones de compilación y de diseño de lenguaje.

Desde Lenguajes de Programación, el aporte propio principal es el sistema de macros. Esta construcción introduce una forma de metaprogramación a nivel de sintaxis: el programador define plantillas de código reutilizable que el compilador expande antes de analizar el programa. Su relevancia no es únicamente sintáctica: modifica el AST, introduce parámetros por valor y de bloque, requiere una estrategia de higiene para evitar captura de variables y exige control de recursión durante la expansión. La construcción `match` acompaña a las macros como azúcar sintáctica para clasificación por tipo dinámico, resuelta mediante desazucarado a `let`, `if` e `is`.

2. Alcance del compilador

2.1. Propósito general

El propósito del compilador es transformar programas escritos en HULK en un bytecode propio y, posteriormente, ejecutarlo sobre una máquina virtual de pila. Para lograrlo, el sistema

no realiza una traducción directa desde el código fuente hasta instrucciones de máquina, sino una sucesión de representaciones con responsabilidades separadas: tokens, AST, AST expandido (sin macros ni `match`), AST verificado y anotado, bytecode y, finalmente, ejecución.

Esta separación permite que cada fase tenga un contrato claro. El frontend debe reconocer la estructura del programa; la expansión debe eliminar las construcciones de azúcar; la semántica debe rechazar programas inválidos y anotar tipos; el generador debe producir bytecode correcto; y la máquina virtual debe ejecutar ese bytecode con un modelo de valores y objetos bien definido.

2.2. Lenguaje soportado

El compilador cubre el núcleo expresivo de HULK y varios mecanismos avanzados. El lenguaje soportado incluye literales numéricos, booleanos y de cadena; operadores aritméticos, booleanos, de comparación y de concatenación (`@`, `@@`); bloques; variables con `let`; asignación destructiva (`:=`); condicionales con `if/elif/else`; ciclos `while` y `for`; funciones globales; llamadas recursivas; objetos con atributos y métodos; herencia simple; despacho dinámico de métodos; `self`; `base`; pruebas dinámicas de tipo con `is`; *casts* con `as`; protocolos estructurales; vectores con literales, indexación y comprensiones; iterables y `range`; lambdas y *closures*; macros con `define`; y expresiones `match`.

La expresión final del archivo funciona como punto de entrada del programa. Este rasgo obliga a tratar condicionales, ciclos, bloques, llamadas y `match` como expresiones con tipo y valor resultante, no solo como instrucciones. Por tanto, el análisis semántico calcula tipos para expresiones compuestas y la generación de código preserva el valor producido por cada construcción.

Listing 1: Programa con funciones, objetos, herencia y despacho dinámico

```
function square(x: Number): Number => x * x;

type Shape() {
  area(): Number => 0;
}

type Circle(r: Number) inherits Shape {
  r: Number = r;
  area(): Number => 3 * square(self.r);
}

let s: Shape = new Circle(4) in print(s.area());
```

2.3. Alcance técnico

El compilador principal está implementado en C++ y se construye con CMake usando Flex para el analizador léxico y Bison para el analizador sintáctico. La estructura física del

repositorio refleja las fases del compilador. Sin embargo, esta división no es el objetivo central del diseño, sino un medio para mantener límites claros entre responsabilidades.

Componente	Responsabilidad principal
<code>src/lexer.l</code>	Especificación Flex del analizador léxico; tokens, literales, comentarios, seguimiento de línea/columna y errores léxicos.
<code>src/parser.y</code>	Gramática Bison (GLR); precedencias, estructura de declaraciones y expresiones, construcción directa del AST.
<code>include/ast.hpp</code>	Jerarquía de nodos del AST y patrón <i>visitor</i> .
<code>src/macro_expander.cpp</code>	Expansión de macros y desazucarado de <code>match</code> (transformación AST→AST).
<code>src/semantic_analyzer.cpp</code>	Resolución de nombres, registro de tipos, chequeo semántico, inferencia acotada y transpilación de lambdas.
<code>src/type_table.cpp</code>	Tabla de tipos, subtipado, conformidad de protocolos y tipos sintéticos (<code>Vector</code> , <code>Iterable</code> , funtores).
<code>src/code_generator.cpp</code>	Generación de bytecode desde el AST verificado.
<code>include/bytecode.hpp</code>	Definición del conjunto de instrucciones y del programa de bytecode.
<code>src/serialize.cpp</code>	Serialización/deserialización binaria del bytecode.
<code>src/vm.cpp</code>	Máquina virtual de pila que ejecuta el bytecode.
<code>src/main.cpp</code> / <code>src/vm_main.cpp</code>	Controladores de <code>hulk</code> y <code>hulk-vm</code> .

2.4. Pipeline de alto nivel

El pipeline completo puede resumirse de la siguiente manera:

Listing 2: Pipeline conceptual del compilador

```

archivo .hulk
-> lexer Flex           # tokens con linea/columna
-> parser Bison GLR     # AST (Program*)
-> MacroExpander        # expande macros, desazucara match
-> SemanticAnalyzer     # 3 pasadas; tipa y transpila lambdas
-> CodeGenerator        # BytecodeProgram (codigo + constantes +
    tablas)
-> serialize -> compiled.asm # bytecode binario (magic "HULK")
-> ./output (script bash)  # ejecuta ./hulk-vm compiled.asm
-> hulk-vm              # deserializa y ejecuta en la VM de pila

```

El diseño se apoya en una idea clásica de compilación: no intentar resolver todos los problemas en una sola representación. El AST es cómodo para semántica, pero demasiado alto nivel para ejecutar; el bytecode es adecuado para una máquina virtual, pero demasiado bajo nivel para razonar sobre tipos. Por eso se introducen fronteras intermedias, y las construcciones de azúcar (macros, `match`, lambdas) se eliminan antes de llegar al bytecode.

El controlador `hulk` distingue las fases mediante *códigos de salida*, lo que permite su uso en correctores automáticos:

Fase	Código	Significado
Léxico	1	Error léxico (<code>lexical_errors_count > 0</code>).
Sintáctico / Macro	2	Error sintáctico o de expansión de macros.
Semántico	3	Error de análisis semántico.
Éxito	0	Se generó <code>compiled.asm</code> y <code>./output</code> .

3. Frontend

El frontend transforma una secuencia de caracteres en una representación estructurada del programa. Esta fase resuelve dos problemas diferentes. El primero es léxico: identificar palabras clave, operadores, delimitadores, identificadores, literales, comentarios y espacios. El segundo es sintáctico: reconocer cómo esos tokens forman declaraciones, expresiones, tipos, protocolos, bloques y patrones.

3.1. Decisión léxica: lexer generado con Flex

El analizador léxico se especifica con Flex en `src/lexer.1`. Esta decisión es adecuada porque el conjunto léxico de HULK no se reduce a identificadores y números. El lenguaje contiene palabras clave (`function`, `define`, `type`, `inherits`, `new`, `self`, `base`, `let`, `if`, `elif`, `else`, `while`, `for`, `protocol`, `interface`, `extends`, `is`, `as`, `match`, `case`, `default`); operadores de varios caracteres (`@@`, `@`, `==`, `!=`, `<=`, `>=`, `:=`, `=>`, `->`); delimitadores; literales numéricos (enteros y flotantes, incluyendo la forma `.5`); literales de cadena con secuencias de escape; comentarios de línea `//`; e identificadores, incluyendo identificadores con prefijo `$` usados por el sistema de macros.

Un lexer manual daría control fino sobre cada caso, pero haría que la definición del lenguaje quedara dispersa en código imperativo. Al usar Flex, la especificación léxica se vuelve declarativa: las reglas que describen tokens se mantienen separadas del código que los consume. Desde el punto de vista teórico, esta decisión conecta la implementación con la idea de autómatas finitos y expresiones regulares para el reconocimiento léxico.

El lexer mantiene un seguimiento explícito de posición mediante las variables globales `line_number` y `column_number`: cada token incrementa la columna según su longitud y los saltos de línea reinician la columna. Esta información se propaga al parser y al AST, lo que permite que todos los diagnósticos posteriores indiquen (`línea, columna`).

Los errores léxicos se reportan con un formato uniforme y se contabilizan en `lexical_errors_count`; entre ellos están la secuencia de escape inválida, la cadena sin cerrar (en fin de línea o de archivo) y el carácter inesperado:

Listing 3: Formato de error léxico

```
(linea,columna) LEXICAL: unexpected character 'X'
(linea,columna) LEXICAL: unterminated string literal
```

3.2. Decisión sintáctica: Bison en modo GLR

El analizador sintáctico se especifica con Bison en `src/parser.y` y se declara `%glr-parser`. La elección de GLR (Generalized LR) en lugar de LALR puro es deliberada: HULK es un lenguaje basado en expresiones con construcciones que generan conflictos de desplazamiento/-reducción difíciles de eliminar por completo en LALR (por ejemplo, lambdas como operandos, o construcciones de control que pueden aparecer como operando derecho de un binario). GLR permite que el generador conserve varias derivaciones cuando una decisión no puede resolverse localmente, y resuelve la mayoría de los conflictos mediante las declaraciones de precedencia y asociatividad.

La gramática define una escalera de precedencia explícita, de menor a mayor poder de enlace: `in` y `else`; asignación `:=`; flechas `=>` y `->`; disyunción `|`; conjunción `&`; negación `!`; comparaciones y `is/as` (no asociativas, lo que impide encadenarlas); concatenación `@/@@`; suma y resta; multiplicación, división y módulo; potencia `^`; y negación unaria. Esta declaración hace explícito el diseño de precedencias y evita multiplicar reglas intermedias.

La estructura general distingue declaraciones de nivel superior (funciones, tipos, protocolos y macros) de la expresión global final. Las expresiones se organizan en una jerarquía descendente (`assign\_expr`, `or\_expr`, ..., `primary\_expr`) coherente con la escalera de precedencia.

Construcciones de control como operando derecho. Una decisión de diseño reciente y relevante es permitir que `if`, `while`, `for` y `match` aparezcan como operando *derecho* de un operador binario, de modo que expresiones como

```
evens := evens + if (i % 2 == 0) 1 else 0;
```

sean válidas. La solución adoptada agrupa estas construcciones en un no terminal `flow\_expr` que *no* es un `primary\_expr` (para que no puedan ser operando *izquierdo*, lo que volvería ambiguo `if (c) x else a + b`) y se añade explícitamente como operando derecho de cada nivel binario. Como cada construcción comienza con una palabra clave distinta, el reconocimiento es determinista y el cuerpo de la construcción se extiende de forma *greedy* hacia la derecha, preservando la semántica esperada. Esta decisión fue necesaria para superar el último caso de la batería de pruebas.

Los errores sintácticos se reportan con un formato uniforme que incluye posición y el token problemático:

Listing 4: Formato de error sintáctico

```
(linea,columna) SYNTACTIC: <mensaje> near '<token>'
```

3.3. AST como frontera entre sintaxis y semántica

El AST (`include/ast.hpp`) es la primera representación estable del lenguaje. El parser construye directamente nodos de AST; no se mantiene un CST explícito separado, sino

estructuras temporales de *parsing* que se convierten en vectores de punteros únicos antes de armar los nodos finales. Esta separación evita que la fase semántica dependa de detalles accidentales de la gramática.

El AST emplea un patrón *visitor* genérico (`Visitor<T>`) con un método `visit` por cada tipo de nodo. Existen alrededor de treinta tipos de nodos, agrupados en: nodos de expresión (`Literal`, `BinaryExpr`, `UnaryExpr`, `VarRef`, `AssignExpr`, `BlockExpr`, `IfExpr`, `WhileExpr`, `ForExpr`, `LetExpr`, `LambdaExpr`, `FuncCall`, `MethodCall`, `MemberAccess`, `NewExpr`, `VectorLiteral`, comprensiones, `VectorIndex`, `ArrayNewExpr`, `ArrayAssignExpr`, `IsExpr`, `AsExpr`, `SelfRef`, `BaseCall`); nodos de declaración (`FuncDef`, `TypeDef`, `AttributeDef`, `MethodDef`, `ProtocolDef`, `MacroDef`); y el nodo raíz `Program`, que agrupa tipos, protocolos, funciones, macros y la expresión global.

Dos nodos tienen un comportamiento especial que documenta una decisión de diseño: `MacroInvoke` y `MatchExpr` lanzan una excepción si se les invoca `accept()`. Esto garantiza, por construcción, que ambos deben haber sido eliminados por la fase de expansión antes de llegar a la semántica o a la generación de código. El compilador no permite, por diseño, que una macro o un `match` sin expandir alcancen fases posteriores.

4. Análisis semántico

Después del frontend y de la expansión, el compilador conoce la estructura del programa, pero todavía no sabe si esa estructura tiene sentido. La fase semántica (`src/semantic_analyzer.cpp`) responde preguntas que la gramática no puede resolver: si una variable está definida, si una llamada tiene la aridad correcta, si un tipo existe, si una condición es booleana, si un método pertenece al receptor, si un *cast* es admisible, si una redefinición respeta la firma del padre o si un tipo satisface un protocolo.

4.1. Análisis en varias pasadas

El análisis se organiza en tres pasadas secuenciales sobre el AST. Esta decisión responde a una necesidad del lenguaje: las declaraciones globales pueden referirse entre sí, las funciones pueden ser recursivas y la herencia requiere conocer la jerarquía completa antes de validar cuerpos.

1. **Registro de tipos.** Recolecta todas las definiciones de tipo, valida ausencia de tipos, atributos, métodos y parámetros duplicados, registra protocolos, comprueba que no existan ciclos de herencia y registra los tipos en orden topológico (los padres antes que los hijos).
2. **Registro de funciones.** Recolecta las funciones globales, valida que no haya nombres ni parámetros duplicados y almacena sus firmas.
3. **Chequeo de tipos.** Verifica cuerpos de métodos, inicializadores de atributos, cuerpos de funciones y la expresión global; acumula errores y, si los hay, los imprime y aborta con código de salida 3.

4.2. Tabla de tipos, subtipado y protocolos

La tabla de tipos (`src/type_table.cpp`) registra los tipos primitivos `Number`, `String`, `Boolean` y el tipo raíz `Object`, además de tipos sintéticos generados bajo demanda: `Vector<T>`, `Iterable<T>` y tipos funtor (`\_FunctorType\_N`) para valores invocables. Los tipos primitivos son *finales*: heredar de ellos es un error.

El subtipado nominal recorre la cadena de padres: todo tipo conforma con `Object`, y un tipo conforma con un ancestro si aparece en su cadena de herencia. Los contenedores son covariantes (`Vector<T> <= Vector<U>` si `T <= U`) y un `Vector<T>` conforma con `Iterable<T>`.

Para protocolos, el sistema valida conformidad *estructural*: un tipo puede usarse donde se espera un protocolo si posee todos los métodos requeridos con firmas compatibles, comprobando además reglas de varianza (contravarianza en parámetros y covarianza en el tipo de retorno). Esta decisión introduce una forma de polimorfismo basada en comportamiento, no en nombres.

4.3. Inferencia acotada

HULK permite omitir anotaciones de tipo en varios contextos, por lo que el compilador realiza una inferencia *acotada* (no un sistema general de inferencia). Las decisiones principales son: los parámetros sin anotar adoptan por defecto el tipo `Number` (heurística justificada porque HULK no tiene sobrecarga de operadores y un parámetro usado aritméticamente solo puede ser numérico); el tipo de un atributo sin anotar se infiere de su inicializador; el tipo de una literal de vector es el ancestro común más bajo (LCA) de sus elementos; el tipo de un `if` es el LCA de sus ramas; y el retorno de una función puede inferirse de su cuerpo. Se trata de una inferencia local de una sola pasada, no de un punto fijo iterativo: un compromiso entre expresividad y simplicidad.

4.4. Resolución de nombres, `self` y `base`

La resolución de nombres usa una tabla de símbolos basada en una pila de ámbitos: ámbito global (con las constantes `PI` y `E` predefinidas), ámbitos de función, ámbitos de `let` y ámbitos de método. Las búsquedas van del ámbito más interno al más externo. `self` solo es válido dentro de métodos y su tipo es el del tipo contenedor; no es un objetivo de asignación válido. `base()` solo es válido dentro de métodos y se resuelve contra el método homónimo del tipo padre, lo que distingue la llamada al ancestro del despacho dinámico ordinario.

4.5. Transpilación de lambdas a funtores

Una decisión central de esta fase es que las lambdas no se tratan como un valor primitivo del runtime, sino que se *transpilan* a un tipo funtor. Al visitar una `LambdaExpr`, el analizador detecta las variables libres capturadas del entorno, genera un `TypeDef` sintético (`\_Lambda\_N`) con un atributo por cada captura y un método `invoke` cuyo cuerpo es el de la lambda, y

anota en el nodo el nombre del tipo generado y la lista de capturas. El cuerpo original se envuelve en un `let` que reenlaza cada variable capturada a `self.\_var`, de modo que el código de la lambda puede seguir refiriéndose a los nombres originales. Esta estrategia reduce la complejidad del backend: una lambda se convierte en `new \_Lambda\_N(capturas)` y su invocación en una llamada al método `invoke`.

4.6. Errores semánticos diferenciados

Una fortaleza de la fase semántica es que distingue errores de naturaleza diferente, todos con el formato `(línea,columna) SEMANTIC: mensaje`. Entre los casos cubiertos aparecen: tipos, funciones, métodos, atributos y parámetros duplicados; símbolos y métodos indefinidos; objetivos de asignación inválidos; aridades incorrectas en funciones, métodos, constructores y `base()`; herencia circular; herencia desde primitivos; padre inexistente; *override* con aridad, firma o retorno incompatibles; atributos privados accedidos desde fuera de su tipo; condiciones no booleanas en `if`, `while` y filtros de comprensión; `for` sobre algo no iterable; operandos inválidos en operadores aritméticos, booleanos, de comparación y de concatenación; *casts* inadmisibles; e inferencia no resuelta. En una defensa académica esto es esencial: un compilador no se evalúa únicamente por los programas que ejecuta, sino también por los programas inválidos que detecta antes de generar código.

4.7. Funciones y constantes incorporadas

El sistema predefine las funciones `print`, `sqrt`, `sin`, `cos`, `exp`, `log` (con base), `rand` y `range`, así como las constantes `PI` y `E`. La función `range` produce un objeto que conforma con el protocolo `Iterable`, lo que permite recorrerlo con `for`.

5. Generación de bytecode

5.1. Motivación de un bytecode propio

Una decisión central del sistema es no emitir ensamblador nativo ni LLVM, sino un *bytecode propio* ejecutado por una máquina virtual. Esta elección responde al alcance del proyecto: generar ensamblador exigiría manejar convenciones de llamada, registros, pila, alineación y detalles de plataforma; integrar LLVM añadiría una dependencia y un nivel de abstracción que no es necesario para los objetivos del curso. Un bytecode propio, en cambio, permite concentrar el trabajo en la traducción semántica de HULK y mantener una representación legible y depurable, cercana a una máquina de pila.

5.2. Modelo de pila y conjunto de instrucciones

El bytecode (`include/bytecode.hpp`) está orientado a una *máquina de pila*: los operandos se empujan y se consumen de una pila de valores. El conjunto de instrucciones cubre movimiento de datos y ámbitos, aritmética, lógica, concatenación, comparación, control de flujo, llamadas, modelo de objetos, vectores, iteración y rangos.

Grupo	Instrucciones representativas
Datos y ámbitos	PUSH_CONST, LOAD, STORE, ASSIGN, POP, BEGIN_SCOPE, POP_SCOPE.
Aritmética y lógica	ADD, SUB, MUL, DIV, POW, MOD, NEG, NOT, AND, OR.
Comparación y cadenas	CMP_EQ, CMP_NEQ, CMP_LT, CMP_GT, CMP_LE, CMP_GE, CONCAT.
Control de flujo	JUMP, JUMP_IF_FALSE, JUMP_IF_TRUE, LABEL, RETURN, HALT.
Funciones	CALL.
Objetos	NEW, GET_ATTR, SET_ATTR, SELF, METHOD_CALL, BASE_CALL.
Tipos dinámicos	IS, AS.
Vectores e iteración	NEW_VECTOR, VECTOR_PUSH, VECTOR_INDEX, VECTOR_STORE, SIZE, ITER_NEXT, ITER_CURRENT, RANGE.

Un `BytecodeProgram` contiene cuatro secciones: la lista de instrucciones, una tabla de constantes (con deduplicación), una tabla de funciones (que mapea nombres a índices de instrucción, incluyendo métodos como `Tipo.metodo` y constructores como `Tipo.\_\_init\_\_`) y una tabla de ancestros por tipo, usada para el despacho dinámico en tiempo de ejecución.

5.3. Lowering de objetos, métodos y base

El generador de código (`src/code_generator.cpp`) construye, para cada `TypeDef`, un método constructor `Tipo.\_\_init\_\_`. Si el tipo tiene padre, el constructor comienza con un `BASE\_CALL` al `\_\_init\_\_` del ancestro; luego inicializa los atributos a partir de los parámetros del constructor y de los inicializadores, y devuelve `self`. La instanciación `new T(...)` emite `NEW T`, evalúa los argumentos y llama `\_\_init\_\_`.

El despacho dinámico se resuelve en tiempo de ejecución: `METHOD\_CALL` construye el símbolo `tipo.metodo` y, si no lo encuentra, recorre la cadena de ancestros del objeto hasta hallar la primera implementación. Esto logra el efecto de una tabla virtual sin almacenar punteros de vtable, lo que simplifica la serialización. La llamada `base(...)` se compila como `BASE\_CALL` estático contra el método del ancestro, evitando que vuelva a invocarse el *override* del hijo.

5.4. Lowering de self, vectores, ciclos y lambdas

El `self` implícito de los métodos se trata como un valor disponible mediante la instrucción `SELF`. Los vectores literales se construyen con `NEW\_VECTOR`; la indexación usa `VECTOR\_INDEX`; la asignación por índice, `VECTOR\_STORE`; y la longitud, `SIZE`. Los ciclos `for` y las comprensiones

se bajan a un patrón basado en `ITER\_NEXT/ITER\_CURRENT` sobre el iterable. Las lambdas, ya transpiladas a tipos funtor por la semántica, se emiten como `NEW \_Lambda\_N`, seguido de la carga de las variables capturadas y la llamada a `\_\_init\_\_`; su invocación se compila como una llamada al método `invoke`.

Listing 5: Esquema de bytecode para un `for`

```
STORE iter_var          # guarda el iterable
LABEL loop_start
LOAD iter_var
ITER_NEXT               # empuja Boolean: hay siguiente?
JUMP_IF_FALSE loop_end
LOAD iter_var
ITER_CURRENT            # empuja el elemento actual
STORE x                 # variable del for
... cuerpo ...
JUMP loop_start
LABEL loop_end
```

5.5. Serialización

El programa de bytecode se serializa en formato *binario* (`src/serialize.cpp`) a un archivo `compiled.asm`. El formato comienza con un número mágico (`0x48554C4B`, las letras “HULK”) y una versión, seguidos de las secciones de código, constantes, tabla de funciones y tabla de ancestros. La máquina virtual deserializa este archivo para reconstruir el `BytecodeProgram`.

6. Máquina virtual de ejecución

6.1. Arquitectura

La máquina virtual (`src/vm.cpp`) es un intérprete de pila. Su estado consiste en una pila de operandos, un puntero de instrucción, una pila de marcos de llamada (cada uno con dirección de retorno, entorno y `self`) y una cadena de entornos para el ámbito de variables. El bucle principal recorre las instrucciones y las ejecuta mediante un *switch* sobre el código de operación.

6.2. Modelo de valores y objetos

El valor de runtime (`include/value.hpp`) es una unión etiquetada con los casos `Number` (un `double`), `String`, `Boolean`, `Object`, `Vector` y `Null`. Los objetos y vectores se gestionan con punteros compartidos. Un `HulkObject` (`include/hulk_object.hpp`) almacena su nombre de tipo, su cadena de ancestros y un mapa de atributos; el despacho de métodos usa esa cadena de ancestros. Los iterables del runtime, como `HulkVector` y `HulkRange`, exponen `next()` y

`current()`, lo que permite que `ITER\_NEXT` e `ITER\_CURRENT` operen uniformemente sobre vectores, rangos y objetos definidos por el usuario que conformen con `Iterable`.

6.3. Llamadas, despacho y builtins

Una instrucción `CALL` busca primero en la tabla de funciones del programa; si no la encuentra, busca en el entorno global una función nativa (los builtins `print`, `sqrt`, `range`, etc., implementados como objetos de función nativa). `METHOD\_CALL` implementa el despacho dinámico recorriendo la cadena de ancestros del receptor, mientras que `BASE\_CALL` salta directamente a la implementación del ancestro. La máquina virtual lanza errores de ejecución ante situaciones como división por cero, índices fuera de rango, *casts* inválidos o métodos inexistentes.

6.4. Por qué la VM no es “otro backend”

Conviene precisar que la máquina virtual no es un backend alternativo al generador de bytecode, sino su consumidor natural. El compilador `hulk` produce el bytecode y un pequeño *script* `./output` que invoca `hulk-vm` sobre `compiled.asm`. Así, el flujo de artefactos es claro: una sola tubería de compilación seguida de una etapa de ejecución desacoplada.

7. Extensión propia de lenguaje: macros y match

7.1. Problema de diseño

HULK ofrece funciones, condicionales y operadores, pero no un mecanismo para *abstraer patrones de código* que el programador repite con pequeñas variaciones, ni para generar código a partir de plantillas. La extensión propia principal de este proyecto es un *sistema de macros sintácticas*: el programador declara con `define` (alias `def`) plantillas que el compilador expande antes del análisis semántico, sustituyendo parámetros por los argumentos de cada invocación.

7.2. Sintaxis y representación

Una macro se declara como una función, pero con `define`, y vive en un espacio de nombres separado (`Program.macros`). Sus parámetros pueden ser *por valor* (`name`) o *sintácticos* (`*name`), estos últimos destinados a recibir un bloque de código. En el AST, una declaración es un `MacroDef` (con `MacroParam` que marca `is\_syntactic`) y cada invocación es un `MacroInvoke`, que puede llevar un argumento de bloque.

Listing 6: Definición y uso de macros

```
define double(x: Number): Number -> x * 2;
define triple(x: Number): Number -> x * 3;
```

```
define six_times(x: Number): Number -> double(triple(x));

{
    if (six_times(3) == 18) print("ok") else print("fail");
}
```

7.3. Expansión, higiene y recursión

La expansión la realiza `MacroExpander` (`src/macro_expander.cpp`) como una transformación $AST \rightarrow AST$ previa a la semántica. El expansor indexa las macros declaradas, recorre el árbol y, al encontrar un `MacroInvoke` (o una llamada cuyo nombre coincide con una macro), valida la aridad, construye un mapa de sustitución parámetro $\rightarrow$ argumento, *clona* el cuerpo de la macro y sustituye.

La *higiene* se resuelve con identificadores con prefijo `$`: dentro del cuerpo de una macro, un nombre como `$tmp` se renombra a un identificador único generado con un contador global (un prefijo `\_mN\_`), de modo que distintas expansiones no colisionan entre sí ni capturan variables del sitio de invocación. La expansión es recursiva: el cuerpo de una macro puede invocar otras macros, que se re-expanden, con un límite de profundidad (64) como salvaguarda frente a recursión infinita. La existencia de pruebas dedicadas (`define\_hygiene`, `define\_nested`, `define\_recursive\_expand`, `define\_loop`, `define\_conditional`, `define\_chain`, `define\_block`) documenta el alcance del sistema.

7.4. La construcción match

La construcción `match` acompaña a las macros como azúcar sintáctica para clasificación por *tipo dinámico*. Su forma es:

Listing 7: Forma general de match

```
match (subject) {
    case TipoA: cuerpo_a;
    case TipoB: cuerpo_b;
    default: cuerpo_por_defecto;
}
```

El expansor desazucara `match` a una combinación de primitivas ya existentes: evalúa el *scrutinee* una sola vez ligándolo a una variable sintética con `let`, y genera una cadena `if/elif/else` en la que cada brazo prueba `variable is Tipo`; el brazo `default` se convierte en el `else` final. Si no hay `default`, se usa un valor por defecto como respaldo. Esta estrategia tiene varias ventajas: no agrega instrucciones nuevas al bytecode, reutiliza las reglas semánticas de `let`, `if` e `is`, y mantiene la máquina virtual independiente de una construcción de patrones.

Listing 8: Esquema conceptual de desazucarado de match

```
let __match0__ = subject in
```

```
if (__match0__ is TipoA) cuerpo_a
elif (__match0__ is TipoB) cuerpo_b
else cuerpo_por_defecto
```

La evaluación única del *scrutinee* es importante: si tuviera efectos secundarios, reevaluarlo por cada brazo cambiaría el significado del programa; por eso el desazucarado lo guarda primero en una variable.

7.5. Comparación con otros lenguajes

El sistema de macros de HULK se inspira en los macros de sustitución sintáctica presentes en la tradición de Lisp y en el preprocesador de C, pero adopta una posición intermedia. A diferencia del preprocesador de C, opera sobre el AST y no sobre texto, por lo que respeta la estructura del lenguaje y puede aplicar higiene; a diferencia de los macros *syntax-rules* de Scheme, su modelo de higiene es más simple (basado en *\$-placeholders* y renombrado por contador) y no implementa *pattern matching* sobre la forma sintáctica de los argumentos. La construcción *match*, por su parte, recuerda al *match* de Rust o al *when* de Kotlin en la idea de seleccionar por tipo dinámico, pero su alcance es deliberadamente menor: solo patrones de tipo y un brazo por defecto, sin *binding* ni *narrowing* explícito, lo que basta para el modelo de objetos con herencia abierta de HULK sin asumir tipos suma cerrados.

8. Característica adicional: comprensión de listas y generadores perezosos

Además del sistema de macros, el compilador incorpora dos características relacionadas con el procesamiento de secuencias: la *comprensión de listas* (construcción declarativa de vectores) y los *generadores perezosos* (secuencias producidas bajo demanda). Ambas comparten el mismo protocolo de iteración, pero difieren en un punto esencial: la comprensión *materializa* un resultado, mientras que un generador *difiere* el cálculo de sus elementos.

8.1. Comprensión de listas

La comprensión de listas permite construir un vector describiendo cómo se genera cada elemento a partir de los elementos de una secuencia fuente, con un filtro opcional. La gramática (*src/parser.y*) reconoce dos formas, que producen los nodos *VectorComprehension* y *VectorComprehensionFilter*:

Listing 9: Comprensión de listas, con y sin filtro

```
let squares = [x * x | x in range(0, 5)] in print(squares);
let evens    = [x | x in range(0, 10) if x % 2 == 0] in print(evens);
```


Semánticamente, el analizador exige que la fuente sea iterable —un `Iterable<T>`, un `Vector<T>` o un tipo que conforme con el protocolo `Iterable`— e infiere el tipo del elemento a partir de ella; en la forma con filtro, además verifica que la condición sea `Boolean`.

El *lowering* de la comprensión es *ansioso* (*eager*): el generador de código (`src/code_generator.cpp`) inicializa un vector vacío con `VECTOR\_INIT` y recorre la fuente con el mismo protocolo de iteración que el `for`, empujando con `VECTOR\_PUSH` cada elemento generado que pase el filtro. El resultado es un `HulkVector` concreto en memoria.

Listing 10: Bytecode de una comprensión con filtro

```
... evalua la fuente ...
STORE _iter
VECTOR_INIT
STORE _vec
LABEL loop_start
LOAD _iter
ITER_NEXT           # hay siguiente?
JUMP_IF_FALSE loop_end
LOAD _iter
ITER_CURRENT
STORE x             # variable de la comprension
... evalua el filtro ...
JUMP_IF_FALSE loop_start # si el filtro es falso, no se empuja
LOAD _vec
... evalua el generador ...
VECTOR_PUSH
STORE _vec
JUMP loop_start
LABEL loop_end
LOAD _vec           # el vector materializado es el valor resultante
```

8.2. Generadores perezosos

La fuente de una comprensión —o de un `for`— no necesita existir completamente en memoria. El protocolo de iteración del lenguaje es *perezoso* y basado en *pull*: un iterable expone los métodos `next()` (avanza y responde si quedan elementos) y `current()` (devuelve el elemento actual), y las instrucciones `ITER\_NEXT` e `ITER\_CURRENT` solicitan *un* elemento a la vez. Así, los valores se calculan en el momento en que se consumen y nunca se almacena la secuencia completa de la fuente.

El lenguaje ofrece una anotación de tipo dedicada para estas secuencias perezosas: el sufijo `*` sobre un tipo. La gramática (`src/parser.y`) desazucara `T*` a `Iterable<T>`, de modo que un parámetro declarado `gen: Number*` acepta cualquier generador perezoso de números. El tipo `Iterable` está predefinido como protocolo con los métodos `next()` y `current()` (`src/type_table.cpp`), y la función incorporada `range` produce precisamente un generador perezoso de este tipo.

Un generador perezoso puede definirse como un tipo ordinario que implemente el protocolo: su estado interno avanza en `next()` y el valor se computa en `current()`. El siguiente ejemplo—tomado de la batería de pruebas— define un generador de cuadrados y lo consume mediante un parámetro de tipo `Number*`:

Listing 11: Generador perezoso y su consumo

```
type Squares(n: Number) {
  i: Number = 0;
  limit: Number = n;
  next(): Boolean { self.i := self.i + 1; self.i <= self.limit; }
  current(): Number { self.i * self.i; }
}

function sum_gen(gen: Number*): Number {
  let s: Number = 0 in {
    for (x in gen) { s := s + x; };
    s;
  };
}

print(sum_gen(new Squares(3)));    // 1 + 4 + 9 = 14
```

En tiempo de ejecución, `ITER\_NEXT` sobre un objeto distingue tres casos (`src/vm.cpp`): si es un `HulkRange` llama a su `next()` nativo; si es un `HulkVector` avanza su iterador interno; y si es un objeto de usuario, busca y ejecuta su método `next()` (recorriendo la cadena de ancestros), e igualmente `ITER\_CURRENT` invoca `current()`. Este despacho uniforme es lo que permite que `for` y las comprensiones operen indistintamente sobre rangos, vectores y generadores definidos por el usuario.

8.3. Relación entre ambas características

La distinción de diseño es clara: un *generador* representa una secuencia perezosa que puede pasarse, recorrerse y consumirse elemento a elemento sin materializarse; una *comprensión* consume una de esas secuencias (perezosa o no) y produce un vector concreto. Dicho de otro modo, la comprensión es un *consumidor ansioso* construido sobre el *protocolo perezoso* de iteración. Esta separación permite componer ambos mundos: una comprensión puede alimentarse de `range` o de un generador de usuario, tomando perezosamente de la fuente solo lo necesario para construir su resultado.

9. Validación y resultados

El compilador se valida contra la batería de pruebas de entrega final, organizada en categorías obligatorias y de bonificación. Cada caso compila un programa con `hulk`, ejecuta `./output` y compara la salida con la esperada; los casos de error verifican además el código de salida y la

categoría del diagnóstico (LEXICAL, SYNTACTIC, SEMANTIC). El sistema supera el **100 %** de las pruebas (**115/115**).

Categoría	Resultado	Cobertura
ok/minimal	20/20	Expresiones, let , control de flujo, funciones.
ok/types	10/10	Tipos, atributos, métodos, retornos.
ok/ooop	10/10	Herencia, despacho dinámico, self , base .
errors/lexical	6/6	Diagnósticos léxicos y código de salida 1.
errors/syntactic	10/10	Diagnósticos sintácticos y código de salida 2.
errors/semantic	15/15	Diagnósticos semánticos y código de salida 3.
ok/macros	8/8	Definición, higiene, anidamiento, expansión recursiva.
ok/interfaces	6/6	Protocolos y conformidad estructural.
ok/arrays	8/8	Vectores, indexación, mutación, recorrido.
ok/lambdas	6/6	Lambdas, <i>closures</i> , composición.
ok/generators	6/6	Iterables definidos por el usuario.
ok/extras	10/10	Casos combinados, incl. control de flujo como operando.

10. Construcción y reproducibilidad

El proyecto se construye con CMake y requiere Flex y Bison. El objetivo de construcción genera ambos ejecutables y los copia a la raíz del repositorio:

Listing 12: Construcción

```
cmake -B build -S .
cmake --build build -j
cp build/hulk . && cp build/hulk-vm .
```

11. Limitaciones y trabajo futuro

El diseño asume varios compromisos conscientes. La inferencia de tipos es local y adopta **Number** como tipo por defecto de parámetros sin anotar, lo que puede ser conservador en programas que usan parámetros no numéricos sin anotación. La higiene de macros se basa en **\$-placeholders** explícitos y no en un sistema completo de higiene automática. La construcción **match** solo admite patrones de tipo y un brazo por defecto, sin *binding* ni patrones estructurales. El despacho dinámico recorre linealmente la cadena de ancestros en lugar de usar tablas virtuales materializadas. Como trabajo futuro se plantean: una inferencia más expresiva, higiene automática de macros, patrones con *binding* y *narrowing* en **match**, y optimizaciones de bytecode (vtables, plegado de constantes).

12. Conclusiones

El compilador de HULK presentado implementa, de extremo a extremo, una tubería de compilación por fases en C++: análisis léxico con Flex, análisis sintáctico GLR con Bison, un AST con patrón *visitor*, una fase de expansión que elimina macros y `match`, un análisis semántico en tres pasadas con sistema de tipos, protocolos estructurales e inferencia acotada, la transpilación de lambdas a tipos funtor, la generación de un bytecode de pila propio y una máquina virtual que lo ejecuta. Cada frontera del compilador responde a una decisión de diseño justificada por las necesidades de las fases vecinas. La extensión propia —un sistema de macros sintácticas con higiene y expansión recursiva, junto con `match`— aporta metaprogramación a nivel de sintaxis sin incrementar la complejidad de las fases de bajo nivel, gracias a su resolución como transformaciones AST→AST. El resultado es un sistema observable por capas que supera la totalidad de la batería de pruebas de entrega final.